

A crew of agents that cleans and explores any dataset

A multimodal exploratory data analysis (EDA) app built with CrewAI. It accepts CSV, Excel, text, images, audio, video, or zips of any of these, then cleans the data, explores it, and has agents check each other's claims. It runs free on a Hugging Face ZeroGPU Space.

Audience Data science professionals on LinkedIn **Status** Design complete, build not started

Date 25 September 2026

Name MOSAIC EDA: Multimodal Orchestrated System for Analysis, Inspection & Cleaning

01 Summary

This project shows skill in agent architecture and orchestration, and it's useful to working data scientists. A user uploads a file or pastes a link. A CrewAI Flow works out what the data is, sends it to specialist crews, cleans it with a fixed set of allowed operations, checks every claim against computed evidence, and returns an interactive report, a PDF, the cleaned data, and a Python script that repeats the cleaning.

CORE DESIGN RULE

Code does the computing. Agents do the interpreting. Every number comes from a tool. Agents plan, judge, and explain. Because each claim links back to a tool output, code can check whether the claim is right.

Decisions made so far

Area	Decision
Name	MOSAIC EDA, short for Multimodal Orchestrated System for Analysis, Inspection & Cleaning. "EDA" is added to tell it apart from Databricks' Mosaic AI. Space and repo: <code>mosaic-eda</code> .
Inputs	Upload or URL. CSV, XLSX/XLS, TXT, JPG/PNG, common audio and video. Zips are the main use case.

Limits	200 MB per input. Up to 500 files sampled per zip, stratified by folder.
Mixed-type zips	The app asks the user: analyze all types together, or only the most common type.
Analysis goal	Optional text box, for example "predict churn". It shapes triage and target-aware checks.
LLM	Google Gemini on the free tier, using two pools of models: a Flash-Lite pool does most of the work and a Flash pool handles the judgment-heavy calls. About 6–8 calls per run. Visitors can add their own key.
Orchestration	A CrewAI Flow is the manager. There's no hierarchical manager agent. Crews are sequential, have fixed handoffs, and don't delegate.
Cleaning	Only operations from an allowed list of about 110. No code written by the LLM ever runs. The app exports <code>cleaning_pipeline.py</code> .
Transcription	Voice-activity check first, then Whisper on a ZeroGPU GPU, with <code>faster-whisper</code> on CPU as fallback. Gemini listens to 1–3 clips for context.
UI	Gradio. A basic UI first, polished later. Icons, never emojis.
Visual design	"Harvest" autumn palette in light and dark, plus a color-blind-safe mode. Highlights SVGs for decoration (117 files, already in <code>assets/decor/</code>) and Lucide for icons.
Reports	Interactive HTML, a PDF, cleaned data, the pipeline script, and a trace log. Stored in a Hugging Face Dataset repo so each report has a link.
Hosting	Free Gradio-SDK Space on ZeroGPU. A Dockerfile stays in the repo for local and portable runs. GitHub Actions deploys to the Space.

02 Scope and inputs

Supported inputs

Type	Formats	Handled by
Table	<code>.csv</code> <code>.tsv</code> <code>.xlsx</code> <code>.xls</code> , plus CSV-shaped <code>.txt</code>	Table crew

Text	<code>.txt</code> (prose, logs, JSON lines)	Text crew. Logs are parsed into a table and sent to the Table crew.
Image	<code>.jpg</code> <code>.jpeg</code> <code>.png</code> , with GIF, WebP and HEIC handled when possible	Image crew
Audio	<code>.mp3</code> <code>.wav</code> <code>.m4a</code> <code>.ogg</code> <code>.flac</code>	Audio crew
Video	<code>.mp4</code> <code>.mov</code> <code>.webm</code> <code>.mkv</code>	Split into sampled frames and an audio track, which go to the Image and Audio crews. A Video Synthesizer merges the results.
Archive	<code>.zip</code> (main case)	Unpacked safely, inventoried, then routed by the file types inside

Where input can come from

- **Direct upload** through Gradio, up to 200 MB.
- **Links:** direct URLs, Google Drive share links, Dropbox share links, Hugging Face Hub file links, and public S3 or GCS URLs. Share links are rewritten to direct-download form, and files are streamed to disk in chunks.

Limits

Limit	Value	Why
Input size	200 MB	Fits the free Space's disk and RAM, and keeps a run under a few minutes
Files analyzed per zip	500, stratified sample	Keeps CPU time and Gemini calls bounded. The report says the analysis used a sample.
Unpacked zip size	2 GB total, 10,000 entries, nesting depth 2	Protects against zip bombs
Image size	60 megapixels decoded	Protects against decompression bombs
Transcription	About 10 minutes of speech per job	Keeps GPU quota and CPU fallback time bounded

Video	First 2 minutes analyzed	Frame sampling and audio stay small
Table rows	Full data read lazily with Polars or DuckDB. Row-heavy stats run on up to 1M rows.	Memory safety on 16 GB

03 System architecture

The app has five layers. Each layer talks only to the ones next to it, so each can be tested on its own.

L1	UI (Gradio)	Upload or link, analysis goal, live agent feed, mixed-zip prompt, optional plan review, report viewer, downloads
L2	Orchestrator (Flow)	Typed state, routers, bounded revision loops, parallel branches, pause and resume, retry budgets
L3	Crews (agents)	Triage, Table, Text, Image, Audio, Video synthesis, Cross-type synthesis, Review, Report
L4	Tools (plain code)	Ingestion, profiling, allowed cleaning operations, statistics, charts, vision, transcription
L5	Storage	Per-job workspace and evidence artifacts, Hugging Face Dataset repo for reports, trace logs

WHY THE FLOW IS THE MANAGER

CrewAI's hierarchical process lets an LLM manager decide who works next. That costs extra calls, is harder to predict, and is harder to debug. Here the routing is plain, testable code in a Flow, and agents reason only inside their own tasks. It's a deliberate, easy-to-explain design choice.

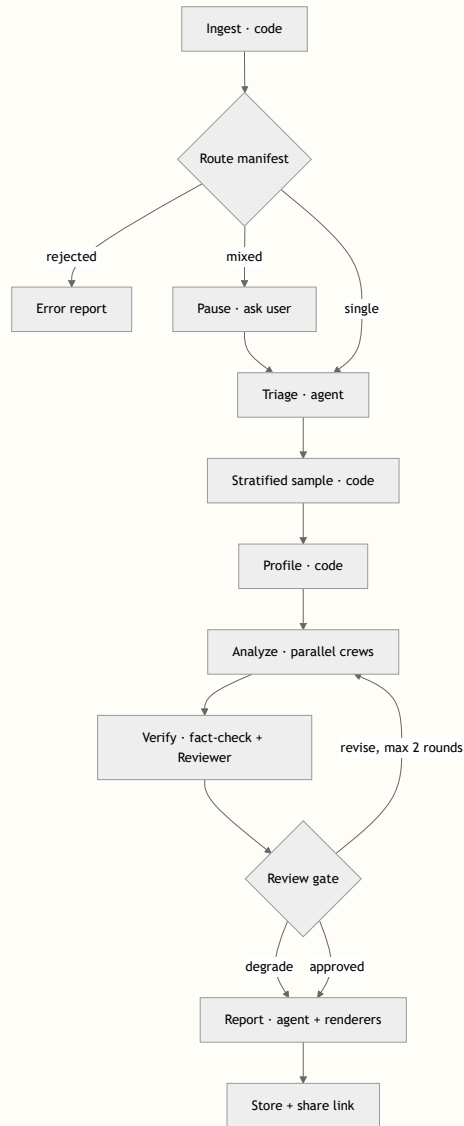
Tech stack

Concern	Choice

Language and runtime	Python 3.12, one of the two versions ZeroGPU supports
Agent framework	CrewAI: Flows, Crews, <code>@CrewBase</code> with YAML agent and task configs, task guardrails, event bus
LLM access	A custom <code>PooledLLM</code> (a CrewAI <code>BaseLLM</code> subclass) that calls Gemini through LiteLLM, with per-model rate limits and automatic fallback. Model lists come from environment variables.
Vision and audio understanding	Google Gen AI SDK (<code>google-generativeai</code>) for contact sheets and sample clips
Data contracts	Pydantic v2
Tables	Polars, DuckDB, openpyxl, xlrd, clevercsv as a fallback for sniffing CSV formats
Text	scikit-learn, spaCy (small English model), langdetect, datasketch, ftfy
Images	Pillow, pillow-heif, OpenCV (headless), imagehash
Audio	ffmpeg, librosa, soundfile, pyloudnorm, webrtcvad-wheels, noisereduce
Transcription	transformers Whisper on ZeroGPU, with <code>faster-whisper</code> on CPU as fallback
Video	ffmpeg and ffprobe, PySceneDetect
Charts	Plotly for the interactive app, matplotlib for the PDF
Reports	Jinja2 templates, WeasyPrint for the PDF
Networking	httpx, streaming downloads with guards
Storage	<code>huggingface_hub</code> uploading to a Dataset repo
UI	Gradio Blocks, custom theme, <code>spaces</code> package for ZeroGPU
Tooling	uv, ruff, pytest, pre-commit, GitHub Actions

04 Orchestration flow

A single `EDAFlow(Flow[EDAState])` runs every job. Steps marked "code" make no LLM calls.



Steps

Step	Kind	What happens
@start ingest	Code	Resolve the URL, stream the download with guards, detect the real file type from its contents, unpack zips safely, and build a <code>FileManifest</code> of files, types, sizes, and folder structure
@router route_manifest	Code	Returns <code>rejected</code> , <code>mixed</code> , or <code>single</code>

Pause for a mixed zip	Code + UI	State is saved with <code>@persist</code> . The UI shows type counts and the Triage agent's hint, and the user picks "group all" or "dominant type only". The Flow resumes from the saved state ID.
<code>triage</code>	Agent	Reads the manifest and the user's goal. Says what the dataset is and what to focus on, and flags things like folders that look like labels or a metadata CSV
<code>sample</code>	Code	Stratified sampling capped at 500 files. Records the sampling ratio for the report.
<code>profile</code>	Code	Runs every profiling and analysis tool for each type ahead of time and saves the results as evidence. No LLM calls.
<code>analyze</code>	Crews	Runs one crew per type, giving each agent the precomputed results in a single request. In group mode, crews run in parallel with <code>asyncio.gather</code> over <code>crew.kickoff_async()</code> , behind the shared <code>PooledLLM</code> rate limits. Video is split first, then its frames and audio reuse the Image and Audio crews.
<code>verify</code>	Code + agent	A code fact-check compares every claimed number with its evidence artifact. Then the Reviewer agent checks the reasoning.
<code>@router</code> <code>review_gate</code>	Code	<code>approved</code> , <code>revise</code> (sends only the affected crew back, with targeted feedback, at most 2 rounds), or <code>degrade</code> (keeps the verified findings and marks the rest unverified)
<code>report</code>	Agent + code	The Report Writer writes the narrative. Code renders the HTML and PDF, exports the cleaned data and pipeline script, and uploads everything to the Hugging Face Dataset repo.

Flow state

```

class EDASState(BaseModel):
    job_id: str
    source: SourceSpec # upload path or URL, user goal, options
    manifest: FileManifest | None = None
    mixed_choice: Literal["group", "dominant"] | None = None
    triage: TriageBrief | None = None
    samples: dict[Modality, SampleSet] = {}
    results: dict[Modality, ModalityResult] = {} # profile, plan, cleaning result, findings
    review: ReviewVerdict | None = None
    revision_round: int = 0 # hard cap: 2
    cleaning_round: dict[Modality, int] = {} # hard cap: 2
    degraded: list[str] = [] # sections shown without narrative

```

```
report: FinalReport | None = None
timings: dict[str, float] = {}
```

Flow skeleton

```
class EDAFlow(Flow[EDAState]):
    @start()
    def ingest(self): ...                                # code only

    @router(ingest)
    def route_manifest(self):
        if self.state.manifest.rejected: return "rejected"
        if self.state.manifest.is_mixed and not self.state.mixed_choice: return "mixed"
        return "single"

    @listen("single")
    def triage(self): ...                                # TriageCrew

    @listen(triage)
    def sample(self): ...

    @listen(sample)
    def profile(self): ...                               # code only, runs every tool first

    @listen(profile)
    async def analyze(self):
        crews = [crew_for(m) for m in self.state.samples]
        await asyncio.gather(*(c.kickoff_async(inputs=...) for c in crews))

    @listen(analyze)
    def verify(self): ...

    @router(verify)
    def review_gate(self): return "approved" | "revise" | "degrade"

    @listen("revise")
    async def revise(self): ...                           # re-runs only the flagged crews

    @listen(or_("approved", "degrade"))
    def report(self): ...
```

The code above sketches the shape of the Flow. Exact decorator wiring will follow the CrewAI version pinned at build time.

05 Agents

Per-type crews

Each type's crew has the same three roles. This keeps the crews consistent and lets them share tests. To stay within the free-tier limits, the Profiler is a code step rather than an agent, and the

other two roles each work from precomputed results in about one call.

Role	Job	Tools	Output
Profiler (code step)	Runs the profiling tools and builds a structured summary of the state of the data. No LLM calls.	Profiling and inventory tools for its type	<code>ProfileReport</code>
Cleaning Strategist	Proposes a cleaning plan using only allowed operations, each with a reason that cites evidence	Operation catalog lookup, dry-run preview	<code>CleaningPlan</code>
Insight Analyst	Analyzes the cleaned data and writes findings that cite evidence	Analysis and chart tools for its type	<code>list[Finding]</code>

Shared agents

Agent	Job	Output
Dataset Triage	Works out what the dataset is and what matters most for the user's goal	<code>TriageBrief</code>
Video Synthesizer	Lines up scene changes with transcript timestamps and merges the visual and audio findings	<code>list[Finding]</code>
Cross-Type Synthesizer	Group mode only. Finds links across types, for example "290 images match rows in metadata.csv, 22 don't"	<code>list[Finding]</code>
Reviewer	A skeptical senior data scientist. Checks reasoning, not arithmetic, using a checklist for each type	<code>ReviewVerdict</code>
Report Writer	Writes the executive summary and section narratives. Never introduces new numbers	<code>ReportNarrative</code>

Agent settings

- `allow_delegation=False` everywhere, so handoffs are fixed, cost is predictable, and traces are readable.
- Tools run first: agents receive precomputed results and evidence IDs in one request. `max_iter=3` allows only occasional follow-up tool calls. Each agent also has a `max_execution_time`.
- Rate limits for each model are enforced by one process-wide limiter, which also covers direct `google-genai` calls.

- `respect_context_window=True` . Agents only ever see summaries, never raw data.
- Reviewer runs at temperature 0. Analysts run at a low temperature.
- CrewAI's built-in memory is turned off. Flow state and the evidence store hold everything agents need, with no embedding costs.
- Agents and tasks live in YAML files (`agents.yaml` , `tasks.yaml`) for each crew, so prompts can change without code changes.

Model plan and call budget

Free-tier limits measured on this account on 25 September 2026. Each model has its own separate limit.

Pool	Models, in fallback order	Limit per model	Pool total	Used for
Lite (main)	<code>gemini-3.5-flash-lite</code> , <code>gemini-3.1-flash-lite</code>	15 per minute, 500 per day	30 per minute, 1,000 per day	Triage, Cleaning Strategist (fallback), Insight Analyst, Report Writer, Video and Cross-Type Synthesizers, vision and listening calls
Flash (judgment)	<code>gemini-3.8-flash</code> , <code>3.7-flash</code> , <code>3.6-flash</code> , <code>3.5-flash</code>	5 per minute, 20 per day	20 per minute, 80 per day	Reviewer. Also the Cleaning Strategist while the pool has more than 30% of its daily total left.

Step	Calls per run
Triage	1
Profiling	0 (code)
Cleaning Strategist	1
Insight Analyst	1–2
Reviewer	1
Report Writer	1
Retries, on average	about 2

Single-type run

about 6–8 (a mixed-type run uses about twice as many)

- **Fallback:** when a model returns a rate-limit error or reaches its daily count, calls move to the next model in its pool, and then to the Lite pool. The trace records every fallback. A run never fails because one model is out of quota.
- **Quota tracker:** counts calls per model per day and resets when Google's daily window resets. The UI shows "Live runs left today". Rate-limit errors are still handled if the count drifts after a restart.
- **Capacity:** about 60 runs a day with the Flash pool reviewing every run, and well over 100 once later runs fall back to Lite only. One live run at a time, with a queue.
- **Fair use:** capacity comes only from the published per-model limits of one key. The app never creates extra keys or projects to get more quota.
- Visitors who add their own key use their own limits and skip the shared quota.

06 How agents communicate

1 · Flow state

The shared record of the job. Crews read from it and the Flow writes their typed results into it.

2 · Typed task outputs

Inside a crew, tasks pass Pydantic objects through `context=[...]`. The Strategist receives an actual `ProfileReport`, not prose.

3 · Evidence store

Tools save full results as artifacts and return a short summary plus an artifact ID. Agents must cite those IDs.

4 · Events to the UI

A `BaseEventListener` on the CrewAI event bus sends agent, task, tool, and guardrail events to a queue for each job. The UI streams that queue.

Evidence store

Each job gets a workspace at `/tmp/jobs/{job_id}/` containing `input/`, `work/`, `artifacts/`, and `out/`. Every tool call writes an artifact such as `tbl_corr_003.json` or `img_hist_007.png`, registered with a type, the producing tool, its parameters, and a hash. The agent gets back a summary of at most 1,500 characters and the ID.

```
class Finding(BaseModel):
    id: str
    modality: Modality
```

```

title: str
statement: str # "Column `income` is right-skewed (skew = 3.2)"
severity: Literal["info", "warning", "critical"]
evidence: list[str] # ["tbl_dist_012"], at least one
claimed_metrics: dict[str, float] # {"skew": 3.2}, checked by code
confidence: Literal["low", "medium", "high"]
scope: Literal["sample", "full"] # says whether a sample was used

class CleaningOp(BaseModel):
    op: str # must exist in the operation catalog
    target: str | list[str] | None # column, file group, or all
    params: dict[str, Any] # checked against the op's schema
    rationale: str
    evidence: list[str] # required for destructive ops

class ReviewVerdict(BaseModel):
    approved: bool
    issues: list[ReviewIssue] # finding_id, problem, fix_request, target crew

```

This design has three benefits. Agents' context stays small. Raw data, and any injection text inside it, doesn't reach the LLM. And code can check every claim.

07 Tools

Every tool is a CrewAI `BaseTool` with a Pydantic `args_schema`, a timeout, and cached results. Tool errors explain how to fix the call, for example "column 'Price' not found; available: price_usd, price_eur".

Ingestion (called by the Flow, not by agents)

Tool	What it does
<code>url_resolver</code>	Converts Google Drive, Dropbox, and Hugging Face share links to direct URLs. Handles Drive's "can't scan for viruses" confirmation page.
<code>safe_downloader</code>	httpx streaming. Allows only http and https. Checks the resolved IP isn't private, loopback, link-local, or a cloud metadata address, and re-checks after redirects. Enforces the size cap and rejects HTML login pages.
<code>type_sniffer</code>	Detects the type from the file's first bytes using <code>filetype</code> , then checks the content (for example, a delimiter test on <code>.txt</code>)
<code>safe_unzip</code>	Blocks path traversal (zip slip). Caps size, entry count, nesting depth, and compression ratio. Skips <code>__MACOSX</code> , <code>.DS_Store</code> , and <code>Thumbs.db</code> . Detects encrypted zips.

`stratified_sampler`

Samples in proportion to each folder, keeps at least a minimum from every folder, and uses a fixed random seed

Table

`load_table` (sheet choice, header row detection, encoding, decimal commas) · `profile_table` · `detect_semantic_types` (ID, currency, percent, date, category, free text) · `missingness_analysis` · `outlier_scan` (IQR and robust z-score) · `association_matrix` (Pearson, Spearman, Cramér's V) · `distribution_summary` · `time_series_checks` · `leakage_check` and `quick_baseline` (HistGradientBoosting with permutation importance, only when a target is known).

Text

`detect_structure` (prose, log, JSON lines, or table) · `text_profile` · `language_mix` · `ngram_frequencies` · `near_duplicate_text` (MinHash) · `topic_model` (NMF) · `entity_extract` (spaCy) · `readability` · `sentiment_sample` (Gemini on a small sample).

Image

`image_inventory` (size, color mode, format, EXIF, corruption) · `quality_metrics` (Laplacian blur, brightness, contrast, near-blank) · `color_stats` · `near_duplicates` (pHash, including cross-class) · `class_balance` (folders as labels) · `contact_sheet` (grids of sample images) · `vision_describe` (Gemini looks at contact sheets rather than 500 separate images).

Audio

`audio_probe` · `standardize` (16 kHz mono WAV) · `signal_stats` (loudness in LUFS, RMS, silence ratio, clipping, estimated SNR) · `speech_detect` (voice activity) · `spectrogram_plot` · `transcribe` (GPU Whisper with CPU fallback) · `music_features` (tempo, chroma) · `listen_describe` (Gemini on 1–3 clips).

Video

`video_probe` · `scene_detect` · `extract_frames` (by fps or by scene) · `extract_audio` · `align_timeline` (scene changes against transcript segments).

Shared

`chart` (Plotly JSON plus a matplotlib PNG from one spec) · `get_artifact` · `fact_check` · `op_catalog_lookup` · `dry_run_plan`.

08 Allowed cleaning operations

There are about 110 operations. Each has a parameter schema, a risk level, a matching code template for `cleaning_pipeline.py`, and a dry-run preview. Operations never change the original data. Each one produces a new version.

Risk level	Meaning	Rule
Safe	Changes format or structure. Values stay the same.	Allowed with a short reason
Lossy	Changes values	Needs a reason. Checked for distribution change afterwards.
Destructive	Removes records or files	Needs a reason with evidence. All destructive ops together may remove at most 30% of records.
View only	Affects analysis only	Not applied to the exported cleaned data

Table (about 45)

- **Structure:** `select_sheet`, `set_header_row`, `combine_multirow_header`, `drop_empty_rows`, `drop_empty_columns`, `standardize_column_names`, `dedupe_column_names`, `split_column`, `unpivot_wide_to_long`, `concat_sheets`
- **Types:** `cast_numeric`, `strip_currency`, `parse_percent`, `fix_decimal_separator`, `parse_dates`, `parse_boolean`, `parse_duration`, `to_category`, `mark_as_id`
- **Missing values:** `standardize_null_tokens`, `impute_median`, `impute_mean`, `impute_mode`, `impute_constant`, `impute_group_median`, `forward_fill`, `add_missing_indicator`, `drop_rows_missing_in` **Destructive**, `drop_high_missing_column` **Destructive**
- **Text in cells:** `trim_whitespace`, `normalize_case`, `normalize_unicode`, `fix_mojibake`, `consolidate_categories` (fuzzy mapping shown to the user), `group_rare_categories`
- **Validity:** `enforce_range`, `validate_pattern` (email, phone, postcode), `fix_sign_errors`
- **Outliers:** `flag_outliers`, `winsorize` **Lossy**, `clip_iqr` **Lossy**, `drop_outliers` **Destructive**
- **Duplicates:** `drop_exact_duplicates` **Destructive**, `drop_duplicates_on_key` **Destructive**
- **Time:** `sort_by_time`, `extract_date_parts`, `flag_time_gaps`
- **Target-aware:** `flag_leaky_column`, `drop_leaky_column` **Destructive**, `log1p_transform` **View only**

Text (about 20)

- **Encoding and format:** `fix_encoding`, `normalize_unicode`, `normalize_whitespace`, `strip_html`, `strip_boilerplate`
- **Structure:** `split_into_documents`, `sentence_segment`, `parse_log_lines` (hands off to the Table crew)
- **Privacy:** `mask_pii`
- **Removing records:** `dedupe_exact`, `dedupe_near`, `drop_short_documents`, `filter_language`, all **Destructive**
- **View only:** `lowercase`, `remove_stopwords`, `lemmatize`

Image (about 22)

- **Format:** `fix_exif_orientation`, `convert_to_rgb`, `flatten_alpha`, `normalize_bit_depth`, `convert_format`, `extract_first_frame`, `standardize_filenames`, `strip_exif`
- **Geometry:** `resize_max_side`, `pad_to_square`, `center_crop`, all **Lossy**
- **Removing files:** `remove_corrupt`, `drop_exact_duplicates`, `drop_near_duplicates`, `drop_too_small`, `drop_blurry`, `drop_too_dark`, `drop_too_bright`, `drop_near_blank`, all **Destructive**
- **Label checks (flag only):** `flag_suspected_mislabeleds`, `flag_cross_class_duplicates`

Audio (about 18)

- **Format:** `convert_to_wav`, `to_mono`, `resample`, `remove_dc_offset`
- **Levels:** `normalize_loudness`, `peak_normalize`, `highpass_filter`, `denoise`, all **Lossy**
- **Segmenting:** `trim_silence` **Lossy**, `split_on_silence`, `segment_fixed_length`
- **Removing files:** `remove_corrupt`, `drop_silent`, `drop_too_short`, `drop_near_duplicates`, `drop_clipped_heavily`, all **Destructive**
- **Flags:** `flag_clipping`, `flag_non_speech`

Video (about 8)

`probe_and_fix_rotation`, `transcode`, `downscale` **Lossy**, `trim`, `sample_frames_fps`, `sample_frames_scene`, `dedupe_frames`, `extract_audio`.

09 Guardrails

Layer	Where	What it does

1 · Input	Ingestion code, before any LLM call	Size, type, pixel, duration, and file-count limits. Blocks private network addresses. Protects against zip bombs and zip slip. Accepts only allowed file types.
2 · Prompt injection	Prompts and tool design	Data reaches agents only as stats or short quoted samples inside <code><untrusted_data></code> tags. Prompts say that content is never instructions. A regex detector flags injection-like text and notes it in the report. The main defense is limited capability: no agent tool writes outside the workspace, reaches the network, or runs code.
3 · Output shape	<code>output_pydantic</code> on every task	Output in the wrong shape is rejected and the agent retries
4 · Output meaning	CrewAI <code>guardrail=</code> functions, with <code>guardrail_max_retries=3</code>	Checks the rules in the table below and returns a specific error message to the agent
5 · Execution	Agent and Flow settings	<code>max_iter</code> , time limits, per-model rate limits and fallback in <code>PooledLLM</code> , tool timeouts, backoff with jitter on 429 errors, and revision caps for each job
6 · Output files	Exporters	Neutralizes CSV formula injection (cells starting <code>= + - @</code>), masks PII in quoted samples, and falls back safely when Gemini's safety filters block a response

Task guardrails (layer 4)

Task	Rejected when
Cleaning plan	An op isn't in the catalog. A target column or group doesn't exist. Parameters fail the schema. A destructive op has no evidence. Estimated row loss is over 30%. Two ops conflict, such as imputing and then dropping the same column.
Findings	A cited artifact doesn't exist. A claimed number differs from its artifact by more than 1%. A finding has no evidence. It uses causal words ("causes", "because of") without support. It claims something about the full data when only a sample was used.

Triage brief	It refers to a file, folder, or column that isn't in the manifest
Report narrative	A number appears that isn't in a verified finding. A required section is missing. It's over the length limit.

```
def validate_findings(output: TaskOutput) → tuple[bool, Any]:
    findings = output.pydantic.findings
    for f in findings:
        if not f.evidence:
            return False, f"{f.id}: add at least one evidence artifact ID."
        for key, claimed in f.claimed_metrics.items():
            actual = evidence.lookup(f.evidence, key)
            if actual is None:
                return False, f"{f.id}: '{key}' not found in {f.evidence}. Cite the artifact that contains it."
            if not close(claimed, actual, rel=0.01):
                return False, f"{f.id}: claims {key}={claimed}, artifact says {actual}. Correct the claim."
    return True, output
```

10 Self-correction

Five levels, ordered from quickest and cheapest to slowest and most thorough. Each loop has a limit stored in state, so none can run forever.

Level	Trigger	Correction	Limit
L1 · Tool feedback	A tool call fails	The error message explains the fix, and the agent corrects its next call	max_iter
L2 · Guardrail retry	Task output fails a guardrail	The specific error goes back to the agent, which tries again	3 per task
L3 · Cleaning check	After the cleaning plan runs, checks find too many rows lost, type conversions failing, new nulls added, or distributions changed too much (KS test)	The router sends the before- and-after comparison to the Cleaning Strategist. If it still fails, only a conservative plan of safe ops is applied.	2 rounds
L4 · Claim check	The code fact-check fails, or the Reviewer raises issues	Only the affected crew reruns, with targeted fix requests	2 rounds

L5 · Fallback	A crew still fails	That section shows the computed stats with no narrative and an "analysis incomplete" label. The job always produces a report.	Final
------------------	--------------------	---	-------

Reviewer checklist (example for tables)

- Were missing values, duplicates, and data types checked before drawing conclusions?
- If there's a target column, was leakage checked? Is class imbalance mentioned?
- Is correlation described as causation anywhere?
- Are claims about a sample presented as true for the whole dataset?
- Is an obvious issue in the profile missing from the findings?
- Does each recommendation follow from a finding?

Every correction is recorded in the trace and shown in the UI's "How the agents checked themselves" tab.

11 Per-type playbooks

Type	Typical cleaning	EDA in the report
Table	Header detection, type fixes, standard null tokens, deduplication, outlier handling, category consolidation	Data quality score, column profiles, missingness patterns, distributions, associations, time checks, leakage and a quick baseline when a target is given
Text	Encoding fixes, boilerplate removal, near-duplicate removal, PII masking, document splitting	Length and vocabulary stats, language mix, n-grams, topics, entities, sentiment on a sample, duplicate clusters
Image	Corrupt-file removal, EXIF rotation, RGB conversion, duplicate removal, quality filters	Class balance, size and aspect distributions, blur, brightness, and contrast, color stats, duplicate pairs (including across classes), suspected mislabels, sample grids, a vision summary

Audio	Standard format, silence trimming, loudness normalization, silent-file removal	Duration, loudness, and silence distributions, clipping, SNR, speech-vs-music split, spectrograms, transcripts analyzed as text, a listening summary
Video	Rotation fix, frame sampling, frame deduplication, audio extraction	Metadata, scene timeline, frame quality, audio results, visual scenes aligned with speech
Mixed (group mode)	Each type cleaned on its own	A section for each type plus cross-type findings, such as metadata that matches or doesn't match the files

12 Edge cases and how they're handled

Ingestion and security

Case	Handling
Google Drive link to a large file	Resolver handles the virus-scan confirmation page. A clear error if the file isn't public.
Dropbox link	Rewritten to <code>?dl=1</code>
Link returns an HTML login page	Detected from content type and file bytes. The user is told to make the file public.
URL points to a private IP or metadata endpoint	Blocked before connecting and again after redirects
URL uses a port other than 80, 443, or 8080	The Space blocks these ports for outgoing traffic, so the app explains the problem instead of timing out
File extension doesn't match the content	Routed by content. For example, a <code>.txt</code> that is really a CSV goes to the Table crew.
Zip bomb, zip slip, deep nesting	Size, ratio, and depth caps. Path checks. Rejected with a reason.
Password-protected zip	Detected and reported. No password prompt in version 1.
Empty zip, or only junk files	Rejected with a list of what was found

Non-UTF-8 file names in a zip	Decoded as CP437 or UTF-8 and normalized
Decompression-bomb image	Pillow pixel limit, and the file is skipped as corrupt
Prompt injection in data or transcripts	Kept in untrusted tags, flagged by the detector, and agents have limited capabilities
Formula injection in exported CSV	Risky leading characters are escaped

Zips

Case	Handling
Mixed file types	The user chooses group or dominant type. Triage adds hints, such as a CSV that looks like metadata.
Folders used as class labels	Detected and used for class balance and cross-class duplicate checks
Very unbalanced folders	Stratified sampling with a minimum per folder. Class balance uses full file counts from the manifest, not the sample.
More than 500 files	Sampled. The report states the sampling ratio. Counts come from the full manifest.
Nested zips	Unpacked up to depth 2, deeper ones listed as skipped

Tables

Case	Handling
Encoding (Latin-1, CP1252, UTF-8 with BOM)	Detected with fallbacks. <code>fix_mojibake</code> is available.
Delimiter <code>;</code> , tab, or <code> </code> , and decimal commas	DuckDB format detection with <code>clevercsv</code> as a fallback. <code>fix_decimal_separator</code> .
Several sheets, header not on row 1, merged headers	Sheet inventory. <code>set_header_row</code> and <code>combine_multirow_header</code> . Old <code>.xls</code> read with <code>xlrd</code> .
Numbers stored as text ("1,200", "12%")	Semantic type detection, then <code>strip_currency</code> or <code>parse_percent</code>

Dates in mixed formats	Tries several formats and reports the parse success rate
Missing values written as "N/A", "- ", "null", "?"	<code>standardize_null_tokens</code>
ID columns that look numeric	<code>mark_as_id</code> , then left out of the stats
Constant, empty, or high - cardinality columns	Flagged, with drop or group ops suggested
1,000+ columns, or larger than memory	Profiling in lazy chunks, column sampling for associations, a summary table first
A single column, or one row	Reduced analysis with an explanation

Text, images, audio, video

Case	Handling
Text: several languages, binary saved as .txt, logs, very large files	Language - mix report. Binary rejected. Logs parsed into tables. Very large text processed in chunks.
Images: CMYK, grayscale, RGBA, 16 - bit	Mode inventory, plus convert, flatten, or normalize ops
Images: EXIF rotation, truncated files, HEIC or WebP, animated GIF	Rotation fixed. Truncated files flagged as corrupt. HEIC via pillow - heif. First GIF frame used.
Gemini safety filter blocks a contact sheet	Retried with a different sample. Otherwise the vision summary is skipped with a note.
Audio: silent or music - only	Voice - activity check skips transcription, and music features run instead
Audio: Whisper inventing text on silence	Only speech segments are transcribed. Low - confidence segments are dropped.
Audio: unusual codecs, broken headers	Converted through ffmpeg. Files that fail are flagged as corrupt.
Video: no audio track, variable frame rate, rotation, static footage	Audio branch skipped. Sampling uses timestamps. Rotation fixed. Near - identical frames removed.

LLM, platform, and runtime

Case	Handling
Gemini 429 rate limit	The call moves to the next model in its pool, then to the Lite pool. Backoff with jitter if every model is busy, with "waiting for quota" shown in the UI.
Both pools used up for the day	Live runs pause with a message and a suggestion to add the user's own key. Example replays still work.
Invalid key entered by a visitor	Checked with one cheap call before the job starts
Agent loops	<code>max_iter</code> and time limits, then the L5 fallback
ZeroGPU quota used up or GPU unavailable	Falls back to <code>faster-whisper</code> on CPU with a smaller audio budget
Space restarts during a job	The job is lost. The UI says the Space restarted and suggests running again.
Several users at once	Gradio queue with a concurrency limit of 1–2 live jobs. Visitors see their queue position.
Temporary files building up	The job workspace is deleted after the report uploads, and a timed sweep clears leftovers

13 External APIs and secrets

Service	Used for	Access	Cost
Google Gemini API (Lite and Flash pools)	All agent reasoning, contact-sheet vision, listening to sample clips	<code>PooledLLM</code> through LiteLLM, and <code>google-genai</code>	Free tier: 1,000 Lite and 80 Flash requests a day across the pools. Visitors can add their own key.
Hugging Face Hub	Storing reports in a Dataset repo and making share links	<code>huggingface_hub</code> with a token scoped to that repo	Free

ZeroGPU	Whisper transcription	<code>@spaces.GPU(duration=...)</code>	Free. Visitors have daily GPU quotas.
Google Drive, Dropbox, public URLs	Reading user inputs	httpx through the safe downloader	Free

Settings and secrets

Name	Kind	Purpose
<code>GEMINI_API_KEY</code>	Space secret (local: <code>.env</code>)	The one key for all agents and all models in both pools. Limits are counted per model, so one key gives the full 1,000 Lite and 80 Flash requests a day.
<code>HF_TOKEN</code>	Space secret	Write access to the reports Dataset repo only
<code>GEMINI_LITE_POOL</code> , <code>GEMINI_FLASH_POOL</code>	Variable	Comma-separated model lists in fallback order, so models can change without redeploying code
<code>FLASH_RESERVE_PCT</code>	Variable	Share of the Flash pool kept for the Reviewer (default 30)
<code>REPORTS_REPO</code>	Variable	For example <code><user>/mosaic-eda-reports</code>
<code>MAX_CONCURRENT_JOBS</code> , <code>GEMINI_RPM</code> , limits	Variable	Tuning without code changes

DATA NOTE SHOWN IN THE APP

This demo uses Gemini's free tier, and Google may use free-tier data to improve its products. Don't upload sensitive data. Files are deleted after each run. Keys that visitors enter are used for their session only and never stored or logged.

14 Report outputs

Output	How it's made

Interactive HTML report	Jinja2 template, Plotly charts, the Harvest or color-blind-safe theme, and links from each finding to its evidence
PDF report	The same template in print layout, rendered by WeasyPrint with matplotlib PNG charts and embedded fonts. A color-blind-safe version is always available.
Cleaned data	A zip of cleaned tables, text, images, or audio, with CSV cells escaped
<code>cleaning_pipeline.py</code>	Built from the code template of each applied operation, so users can rerun it on the full dataset
<code>trace.json</code>	Every event, decision, guardrail rejection, and revision
Share link	The HTML and PDF are uploaded to the Hugging Face Dataset repo under the job ID

Report sections

- 1. Executive summary and data quality score (0–100, with a breakdown)
- 2. Dataset overview: manifest, sampling ratio, what triage concluded
- 3. Top findings, sorted by severity, each with its evidence
- 4. A section for each type
- 5. Cleaning log with risk levels and before-and-after comparison
- 6. How the agents checked themselves
- 7. Recommended next steps
- 8. Appendix: method, limits, versions

15 User interface

Built with Gradio Blocks. Version 1 is basic, and polish comes later. Icons come from Lucide and decoration from Highlights (CC0). There are no emojis anywhere.

Screens

Screen	Contents
Input	Upload and Link tabs, optional goal box, collapsible settings (own Gemini key, sample size, quick or thorough depth, a toggle to review the cleaning plan), an Analyze button, and example buttons

Mixed - zip choice	Type counts, the Triage hint, and two buttons: "Analyze everything together" and "Dominant type only"
Live run	Left: pipeline steps with status and live counters (LLM calls, tool runs, guardrail catches, self - corrections, verified facts). A "Live runs left today" counter from the quota tracker. Right: the agent room, a <code>gr.Chatbot</code> feed with collapsible tool calls and colored markers for rejections and fixes.
Plan review (optional)	An editable table of ops (target, risk, reason, estimated impact, include) with an Approve button
Results	Tabs: Summary · one tab per type present · Cleaning (with pipeline code) · How the agents checked themselves · Downloads · How it works

Replay mode for examples

Example buttons replay recorded real runs (saved events and final outputs) at a watchable speed, marked "Replay of a live run". The demo works even when quotas are used up or the Space has just woken, costs nothing, and gives a clean run to record for LinkedIn. Uploads and links always run live.

Settings remembered in the browser

Color mode (standard or color-blind safe) and theme are stored with `gr.BrowserState` . The visitor's own API key is never kept after the session.

16 Visual design

Harvest palette (default)

Parchment
#FBB6EE

Espresso
#2B1D14

Rust
#B5471B

Mustard
#E8B04A

Spruce
#2F6F73

Chestnut, dark mode
#1C1512

Pumpkin, dark mode
#E0763A

Role	Light	Dark

Background / surface / border	#FBF6EE / #FFFDF8 / #E4D5BF	#1C1512 / #261D18 / #4A3A2E
Text / muted text	#2B1D14 (15.2:1) / #6B5646 (6.4:1)	#F3E9DC (15.0:1) / #BFAE9C (8.4:1)
Accent	#B5471B , white text 5.4:1	#E0763A , dark text 5.9:1
Critical / warning / info / self-corrected	#9E2B25 / #8A5A12 / #3F6475 / #4F6B2F	#E8776C / #E6B14A / #8DB3C4 / #A3C170
Chart colors	rust, spruce, mustard #A57A00 , plum, moss, slate, terracotta, walnut. All at least 3:1.	Lighter versions, all at least 5.7:1
Sequential / diverging scales	cream → rust → espresso / spruce ↔ cream ↔ rust	

Color-blind-safe mode

Simulation (Machado 2009) showed Harvest's chart colors run together for deuteranopia (the closest pair is ΔE 5.5), while Okabe-Ito stays at ΔE 16 or more for all three types of color blindness. Turning on the eye toggle, or opening a link with `?palette=cvd` , switches:

- Chart categories to Okabe-Ito, adjusted for contrast on both backgrounds
- Sequential scales to cividis and diverging scales to blue ↔ orange
- The accent to vermillion
- The PDF to the matching palette

Status never relies on color alone, in either mode. Each status has a warm or cool color family, a distinct icon, a solid or outline badge, and always a text label. Charts label data directly, use marker shapes and dash styles, and group more than six categories.

Typography

Fraunces for headings, Inter for body text, JetBrains Mono for code, tool names, and numbers. All are served from the Space and embedded in the PDF.

Decoration

The Highlights pack is in `assets/decor/` : 117 SVGs in 12 folders (arrows, underlines, loops, blobs, whirls, scribbles, lines, donuts, spirals, sprinkles, punctuation, doodles). The files use a

fixed blue fill (`#4633FF`), which a build step replaces with `currentColor` . Stray `.DS_Store` files are ignored. Five to seven touches in the UI: a headline underline, an arrow pointing to the examples, a circle around the quality score, a highlighter mark behind the top finding, sketch arrows on the How it works page, and one mark on the PDF cover. They use `currentColor` so they follow the theme. None appear in data areas.

17 Deployment

HOSTING ROUTE

A free Gradio-SDK Space on ZeroGPU. Docker Spaces and standard CPU Gradio Spaces now need a PRO plan (\$9 a month). Free accounts in good standing can host up to 2 ZeroGPU Spaces, and the account's eligibility has been confirmed.

Space configuration

```
---
title: MOSAIC EDA
sdk: gradio
sdk_version: <pinned at build time>
python_version: "3.12"
app_file: app.py
license: mit
short_description: Agents that clean, explore, and fact-check any dataset
---
```

File	Contents
<code>requirements.txt</code>	Pinned Python dependencies, including <code>spaces</code> , <code>crewai</code> , <code>gradio</code> , <code>torch</code> , and <code>transformers</code>
<code>packages.txt</code>	System packages: <code>ffmpeg</code> , <code>libpango-1.0-0</code> , <code>libpangoft2-1.0-0</code> , <code>libharfbuzz0b</code> , <code>fonts-noto-core</code> (for non-Latin PDF text)
<code>app.py</code>	Builds the Gradio app, loads the Whisper model onto <code>cuda</code> at import time (as ZeroGPU recommends), configures the queue, and launches

ZeroGPU usage

```
asr = pipeline("automatic-speech-recognition", model=WHISPER_ID, device="cuda") # module level

def _gpu_seconds(segments): return min(10 + 0.1 * total_seconds(segments), 90)
```

```
@spaces.GPU(duration=_gpu_seconds)
def transcribe_gpu(segments): return asr(segments, return_timestamps=True)

def transcribe(segments):
    try: return transcribe_gpu(segments)
    except (QuotaExceeded, GPUUnavailable): return transcribe_cpu_faster_whisper(segments)
```

Only transcription uses the GPU. Everything else runs on CPU and doesn't count against visitors' GPU quota (2 minutes a day for anonymous visitors, 5 minutes for free accounts).

Continuous deployment

1. Code lives in a public GitHub repo.
2. `ci.yml` runs ruff, pytest, and the palette accessibility test on every push and pull request.
3. `sync-to-hf.yml` pushes `main` to the Space with an `HF_TOKEN` GitHub secret. The Space rebuilds on each push.

Local and portable runs

A `Dockerfile` (python:3.12-slim, a non-root user with UID 1000, port 7860) runs the same app anywhere with `docker run -p 7860:7860 --env-file .env mosaic-eda`. Outside ZeroGPU, the `@spaces.GPU` decorator does nothing and transcription uses the CPU path.

Platform facts that affect the design

Fact	Consequence
Disk isn't persistent	Workspaces are temporary. Reports go to the Dataset repo.
Outgoing traffic only on ports 80, 443, 8080	Links on other ports get a clear error
Free Spaces sleep when idle	The first visit after sleep waits for startup. Replay examples load right after.
Public Space means public source code	No hard-coded keys. Secrets are set only in Space settings.
ZeroGPU supports only the Gradio SDK, and PyTorch best	Whisper runs through transformers on the GPU. <code>faster-whisper</code> is only for CPU.
Heavy dependencies	Builds take several minutes. Keep dependencies lean and pinned.

18 Repository layout

```
mosaic-eda/
├─ app.py                                # Space entry point
├─ README.md                            # Space config header + project README
├─ requirements.txt · packages.txt · pyproject.toml · Dockerfile · .env.example
├─ .github/workflows/ ci.yml · sync-to-hf.yml
├─ src/mosaic/
│   ├─ config.py                        # settings, limits, environment variables
│   ├─ flow/                            eda_flow.py · state.py · routers.py
│   ├─ crews/
│   │   ├─ triage/ table/ text/ image/ audio/ video/ synth/ review/ report/
│   │   │   └─ crew.py · config/agents.yaml · config/tasks.yaml
│   └─ models/                          manifest.py · profile.py · cleaning.py · findings.py · review.py ·
report.py
│   ├─ tools/                          ingest/ · table/ · text/ · image/ · audio/ · video/ · shared/
│   └─ ops/                            catalog.py · table_ops.py · text_ops.py · image_ops.py · audio_ops.py ·
video_ops.py · codegen.py
│   ├─ guardrails/ input_guards.py · injection.py · task_guardrails.py · output_guards.py
│   ├─ evidence/                       store.py · fact_check.py
│   ├─ llm/                            pooled_llm.py · quota_tracker.py · rate_limiter.py · key_check.py
│   ├─ gpu/                            whisper.py
│   ├─ reporting/                      html.py · pdf.py · charts.py · templates/
│   ├─ storage/                        hf_dataset.py · cleanup.py
│   ├─ events/                         listener.py · replay.py
│   └─ ui/                             layout.py · theme.py · palettes.py · components.py · handlers.py
├─ assets/                             decor/ (Highlights SVGs) · icons/ (Lucide) · fonts/ · LICENSES.md
├─ examples/                           datasets/ · recordings/
├─ eval/                               golden/ · run_eval.py · results.md
└─ tests/                              unit/ · integration/ · test_palettes.py
```

19 Build plan

The plan builds a thin end-to-end slice first and deploys it early, then adds data types and depth. Each phase ends with something working.

Phase 0 Foundation

- Repo, uv, ruff, pytest, pre-commit, CI
- Config, logging, job workspaces
- `PooledLLM` with quota tracker, tested with the real key from `.env`
- An empty Space with sync from GitHub

Done when a "hello" Gradio app deploys automatically from GitHub.

Phase 1

Ingestion and input safety

- URL resolvers, safe downloader, type sniffer, safe unzip, manifest, sampler
- Tests for every security case

Done when the malicious-input test suite passes.

Phase 2

Table slice, end to end

- Table tools and ops
- Table crew, Flow skeleton, evidence store
- Basic UI with the live feed, basic HTML report

Done when a messy CSV produces a report on the live Space.

Phase 3

Checking and self-correction

- Task guardrails, fact-check, Reviewer crew
- Review gate and revision loops, cleaning checks, fallback mode
- Trace export

Done when a planted wrong number is caught and fixed, visibly, in the UI.

Phase 4

Image crew

- Inventory, quality checks, pHash, class balance, contact sheets, Gemini vision
- Image ops, gallery views

Done when the planted-duplicates image zip is fully diagnosed.

Phase 5

Audio crew and ZeroGPU

- ffmpeg standardizing, signal stats, voice-activity check
- Whisper on GPU with CPU fallback, listening summary, audio ops

Done when the podcast-clips example works on the Space's GPU and in local CPU mode.

-
- Phase 6** **Text crew**
- Structure detection, log-to-table handoff, profiling, topics, entities, near-duplicates, text ops
- Done when** text corpora, logs, and transcripts all flow through.
-
- Phase 7** **Video and mixed zips**
- Video splitting, scene detection, timeline alignment
 - Mixed-zip pause and resume, parallel crews, Cross-Type Synthesizer
- Done when** the mixed-zip example runs in group mode.
-
- Phase 8** **Outputs**
- PDF via WeasyPrint, cleaned-data export, `cleaning_pipeline.py` generation
 - Hugging Face Dataset storage and share links
- Done when** the exported pipeline script reproduces the cleaned data.
-
- Phase 9** **UI polish**
- Harvest theme, color-blind mode, Highlights and Lucide, fonts
 - Replay mode, example datasets, plan review, How it works page
- Done when** the palette tests pass and replays work offline.
-
- Phase 10** **Evaluation and launch**
- Golden datasets and evaluation report
 - README with architecture diagram, demo video, LinkedIn post
- Done when** the evaluation numbers are published in the README.
-

20 Testing and evaluation

Level	What's covered
Unit	Every tool and op (including checks that each op's generated code matches what it did), guardrail functions, resolvers, input safety checks

Security	Zip bombs, zip slip, private-network URLs, decompression bombs, injection text, CSV formula cells
Crew contract	Recorded or mocked LLM replies to test routing, retries, and fallbacks without spending quota
Integration	Full Flow runs on small fixtures for each type, using the real Gemini API, run manually or nightly
Accessibility	Palette simulation: chart colors must stay at ΔE 15 or more for each type of color blindness, and text must meet WCAG AA
Evaluation	About 10 golden datasets with planted problems. Measures detection recall, false alarms, fact-check pass rate, revisions per job, time, and LLM calls.

Golden datasets (examples)

- A sales CSV with a leaky column, mixed date formats, "\$" amounts, and 3% duplicates
- A semicolon-separated European CSV with decimal commas, stored as `.txt`
- A three-class image zip with cross-class duplicates, 5 corrupt files, and one mislabeled folder
- Audio clips including one silent file, one clipped file, and one music-only file
- A text corpus with near-duplicates, boilerplate, and a hidden injection string
- A video with no audio track, plus one with rotation metadata
- A mixed zip of images plus a metadata CSV with 22 unmatched rows

21 Observability

- A structured JSON trace for each job from the event listener: agent steps, tool calls with timings, guardrail results, revisions, token and call counts.
- The trace can be downloaded from the UI and powers replay mode.
- Optional Langfuse tracing (free tier), turned on with environment variables.
- Space logs record job start, end, and failure, but never file contents or keys.

22 Risks and mitigations

Risk	Mitigation

Gemini free-tier limits (Flash models allow only 20 requests a day each)	Tools-first design with about 6–8 calls per run, two model pools with fallback, a quota tracker with runs left shown in the UI, replay examples, visitor-supplied keys, a job queue
Free-tier terms or model names change	Model name set by environment variable. The LLM layer goes through CrewAI's model strings, so switching providers is a config change.
Hugging Face changes its free hosting (it has before)	The Dockerfile keeps the app portable. Replays and the README video keep the demo viewable.
ZeroGPU compatibility problems	GPU use is limited to one function, with a tested CPU fallback
Slow CPU on large inputs	Sampling, lazy loading, time budgets, a quick-depth mode
Agents inventing insights	Evidence-linked claims, code fact-check, Reviewer, fallback mode
Prompt injection through data	Untrusted-data wrapping, detector, agents with no side-effect tools
Dependency conflicts (crewai, torch, gradio)	Pinned versions, CI build check, lean requirements
Long build times	Few system packages, cached wheels, only small models where possible

23 Open items

- **Name:** decided. MOSAIC EDA.
- **Palette:** Harvest confirmed as the default, with the color-blind-safe toggle.
- **Highlights assets:** done. 117 SVGs are in `assets/decor/`.
- **Gemini:** key created and free-tier limits recorded (see the model plan in section 05). Recheck the limits before launch, because Google can change them.
- **Local key:** paste the Gemini key into the project's `.env` file (already created and excluded from git).
- **Check at build time:** the ZeroGPU Space's CPU and RAM, the current Gradio SDK version, and the CrewAI version and its Flow and guardrail APIs.

